



서울 랜드마크 이미지 분류 과제



C팀
박수빈
김연수
김소연

1	
2	
3	
4	
5	
6	
...	

서울 랜드마크 이미지 분류 프로젝트

데이터셋 구조

서울 랜드마크 10개 클래스를 분류하는 CNN 및 전이학습 기반 이미지 분류 모델의 학습 데이터 구조와 전처리 파이프라인을 정리합니다.

데이터셋 불러오기

```
BASE_DIR = '/content/drive/MyDrive/dataset'
TRAIN_DIR = os.path.join(BASE_DIR, 'train')
TEST_DIR = os.path.join(BASE_DIR, 'test')

train_df = pd.read_csv(os.path.join(BASE_DIR, 'train.csv'))
test_df = pd.read_csv(os.path.join(BASE_DIR, 'test.csv'))
sample_submission = pd.read_csv(os.path.join(BASE_DIR, 'sample_submission.csv'))

print('=== train.csv ===')
print(train_df.head())
print(f'총 학습 데이터: {len(train_df)}개')
print(f'클래스 수: {train_df["label"].nunique()}개')
print('=== test.csv ===')
print(test_df.head())
print(f'총 테스트 데이터: {len(test_df)}개')
```

=== train.csv ===

	file_name	label
0	001.PNG	9
1	002.PNG	4
2	003.PNG	1
3	004.PNG	1
4	005.PNG	6

총 학습 데이터: 723개
클래스 수: 10개

=== test.csv ===

	file_name
0	001.PNG
1	002.PNG
2	003.PNG
3	004.PNG
4	005.PNG

총 테스트 데이터: 199개

데이터 분할

구분	Train	Test
용도	학습 및 검증	최종 예측
라벨	있음 (0~9)	없음
이미지 수	723장	199장
학습/검증	578 / 145	—

데이터셋 구조

```
class CustomDataset(Dataset):
    def __init__(self, df, transform=None, is_test=False):
        self.df = df
        self.transform = transform
        self.is_test = is_test

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.df.loc[idx, 'file_path']
        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        if self.is_test:
            return image

        label = self.df.loc[idx, 'label']
        return image, label

print('CustomDataset 정의 완료')
```

```
BATCH_SIZE = 32
```

```
train_dataset = CustomDataset(train_data, transform=train_transform)
val_dataset = CustomDataset(val_data, transform=val_transform)
test_dataset = CustomDataset(test_df, transform=val_transform, is_test=True)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
```

```
print(f'train_loader: {len(train_loader)}개 배치')
print(f'val_loader: {len(val_loader)}개 배치')
print(f'test_loader: {len(test_loader)}개 배치')
```

```
images, labels = next(iter(train_loader))
print(f'배치 이미지 shape: {images.shape}')
print(f'배치 라벨 shape: {labels.shape}')
```

```
train_loader: 19개 배치
val_loader: 5개 배치
test_loader: 7개 배치
```

```
배치 이미지 shape: torch.Size([32, 3, 224, 224])
배치 라벨 shape: torch.Size([32])
```

10 클래스

서울 랜드마크 각 1개

80 : 20

학습/검증 비율

224×224

입력 Shape (3, 224, 224)

Resize(224×224) + `.convert('RGB')`로 통일된 입력
형식 적용.

기본 CNN 모델 구조

```
class CNNModel(nn.Module):

    def __init__(self, num_classes):

        super(CNNModel, self).__init__()

        # 특징 추출 부분
        self.features = nn.Sequential(

            # Conv Block 1
            nn.Conv2d(
                in_channels=3,
                out_channels=32,
                kernel_size=3,
                padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(2),
```

Conv
Block1

```
        # Conv Block 2
        nn.Conv2d(
            in_channels=32,
            out_channels=64,
            kernel_size=3,
            padding=1
        ),
        nn.ReLU(),
        nn.MaxPool2d(2),

        # Conv Block 3
        nn.Conv2d(
            in_channels=64,
            out_channels=128,
            kernel_size=3,
            padding=1
        ),
        nn.ReLU(),
        nn.MaxPool2d(2)
    )
```

Conv
Block2

Conv
Block3

```
        # 분류 부분
        self.classifier = nn.Sequential(

            nn.Flatten(),

            nn.Linear(128 * 28 * 28, 256),

            nn.ReLU(),

            nn.Dropout(0.5),

            nn.Linear(256, num_classes)
        )

    def forward(self, x):

        x = self.features(x)

        x = self.classifier(x)

        return x
```

classifier

기본 CNN-학습 결과

100%|██████████| 19/19 [02:45<00:00, 8.71s/it]

Epoch [1/10]

Train Loss: 2.5199

Validation Accuracy: 0.4207

100%|██████████| 19/19 [00:14<00:00, 1.28it/s]

Epoch [2/10]

Train Loss: 1.4218

Validation Accuracy: 0.6000

100%|██████████| 19/19 [00:15<00:00, 1.24it/s]

Epoch [3/10]

Train Loss: 0.8750

Validation Accuracy: 0.7310

100%|██████████| 19/19 [00:15<00:00, 1.19it/s]

Epoch [4/10]

Train Loss: 0.7909

Validation Accuracy: 0.7793

100%|██████████| 19/19 [00:16<00:00, 1.16it/s]

Epoch [5/10]

Train Loss: 0.5520

Validation Accuracy: 0.8207

100%|██████████| 19/19 [00:16<00:00, 1.19it/s]

Epoch [6/10]

Train Loss: 0.4454

Validation Accuracy: 0.8207

100%|██████████| 19/19 [00:15<00:00, 1.19it/s]

Epoch [7/10]

Train Loss: 0.5119

Validation Accuracy: 0.8690

100%|██████████| 19/19 [00:16<00:00, 1.17it/s]

Epoch [8/10]

Train Loss: 0.4193

Validation Accuracy: 0.8690

100%|██████████| 19/19 [00:16<00:00, 1.17it/s]

Epoch [9/10]

Train Loss: 0.3402

Validation Accuracy: 0.8621

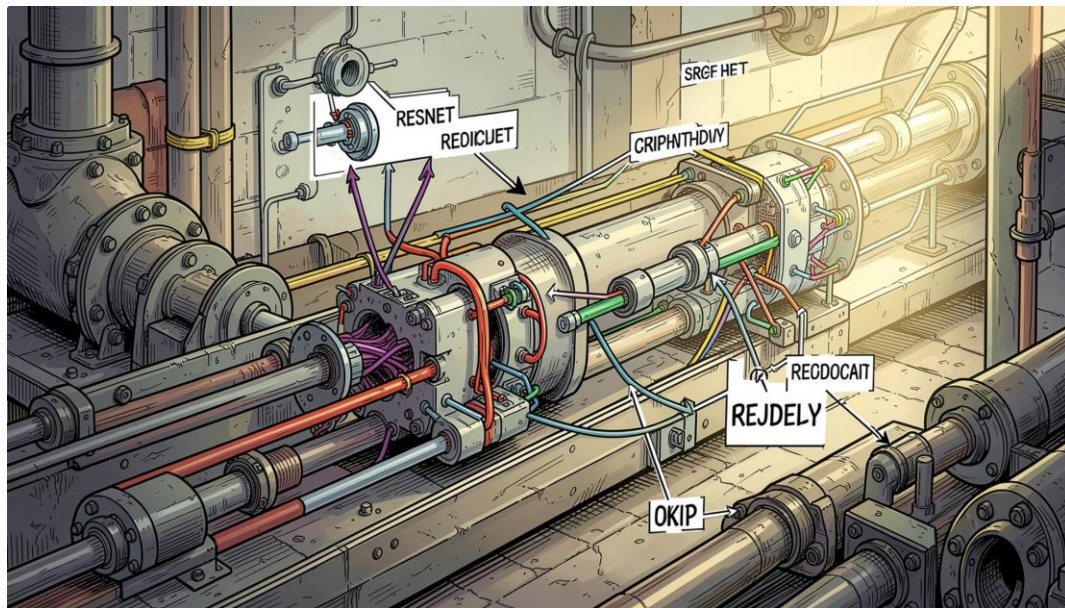
100%|██████████| 19/19 [00:16<00:00, 1.18it/s]

Epoch [10/10]

Train Loss: 0.4490

Validation Accuracy: 0.8759

전이학습 모델 구조 — ResNet18



ResNet18 파이프라인

- ImageNet 사전학습 가중치 로드
- ResNet18 Backbone
잔차 연결로 기울기 소실 방지
- 마지막 FC Layer 교체
1,000 → 10 클래스

두 가지 전략 비교

전략	Freeze	LR
Feature Extr action	Backbone 전체	0.0001
Fine-tuning	없음 (전체 학습)	0.0001

전이학습 모델 구조 — ResNet18

전이학습 모델 정의(14)

```
1 # 8. torchvision 모델 선택(ResNet18 불러오기)
2 model_fe = models.resnet18(pretrained=True)
3 model_ft = models.resnet18(pretrained=True)
4
5 # 6. 마지막 classifier layer 수정 (1000 → 10 클래스)
6 model_fe.fc = nn.Linear(512, 10)
7 model_ft.fc = nn.Linear(512, 10)
8
9 # Feature Extraction - backbone freeze
10 for param in model_fe.parameters():
11     param.requires_grad = False
12 for param in model_ft.fc.parameters():
13     param.requires_grad = True
14
15 print("모델 정의 완료")
```

... === Feature Extraction ===

Epoch 1		Train Loss: 0.3797		Val Loss: 0.2736		Val Acc: 96.55%
Epoch 2		Train Loss: 0.3289		Val Loss: 0.2686		Val Acc: 96.55%
Epoch 3		Train Loss: 0.3311		Val Loss: 0.2607		Val Acc: 95.86%
Epoch 4		Train Loss: 0.3556		Val Loss: 0.2627		Val Acc: 96.55%
Epoch 5		Train Loss: 0.3971		Val Loss: 0.2468		Val Acc: 97.24%

=== Fine-tuning ===

Epoch 1		Train Loss: 0.1567		Val Loss: 0.0515		Val Acc: 98.62%
Epoch 2		Train Loss: 0.0434		Val Loss: 0.0468		Val Acc: 98.62%
Epoch 3		Train Loss: 0.0621		Val Loss: 0.0261		Val Acc: 99.31%
Epoch 4		Train Loss: 0.1307		Val Loss: 0.0413		Val Acc: 98.62%
Epoch 5		Train Loss: 0.0191		Val Loss: 0.0407		Val Acc: 99.31%

실험 설계

세 가지 실험을 통해 CNN 단독, Feature Extraction, Fine-tuning의 성능 차이를 체계적으로 비교합니다. Batch size 32, Optimizer Adam은 공통 적용.

실험	모델	Pretrained	Freeze	LR	Augmentation
Exp1	CNN	X	X	0.001	HorizontalFlip, Rotation(10°)
Exp2	ResNet18	✓	전체	0.0001	동일
Exp3	ResNet18	✓	없음	0.0001	동일

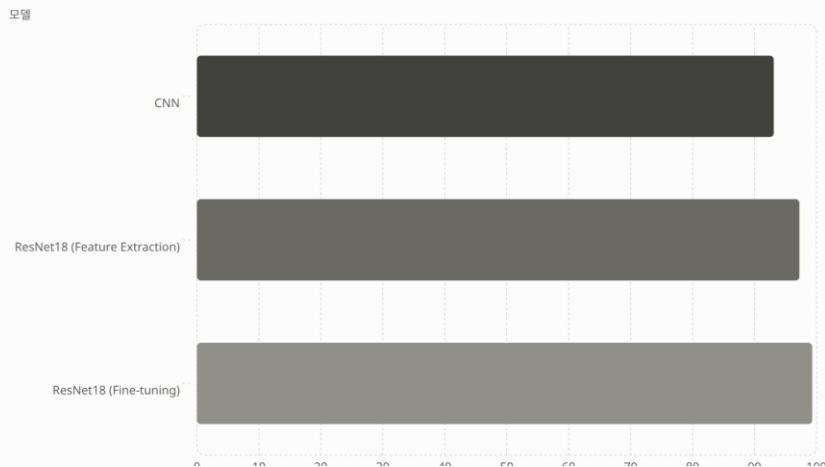
Augmentation 효과

적은 데이터(578장)에서 과적합 억제

Pretrained 결정적 역할

pretrained=False 시 Val Acc 33~47% → 사전학습 가중치의 중요성 확인

모델 성능 비교



Loss 곡선 분석

CNN

Train Loss ↓ / Val Loss 상
대적 높음 → **과적합 징후**

Feature Extraction

Val Loss 전 구간 꾸준히
감소 → **안정적 학습**

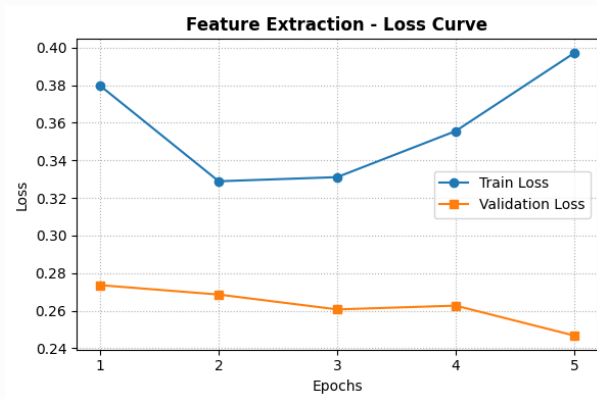
Fine-tuning

Val Loss 0.02~0.05 유지, 가장 빠른 수렴 → **세 모델 중 최고 성능**

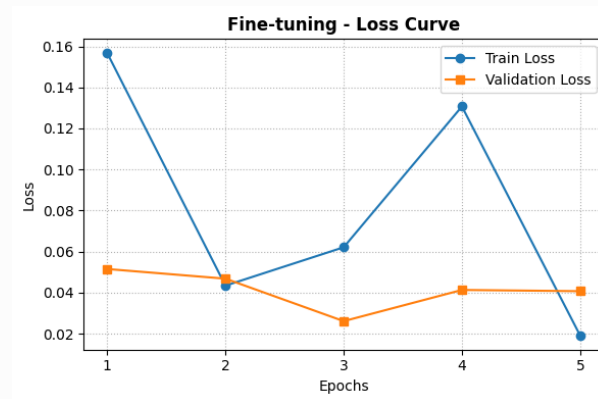
모델 성능 비교 — Loss



CNN

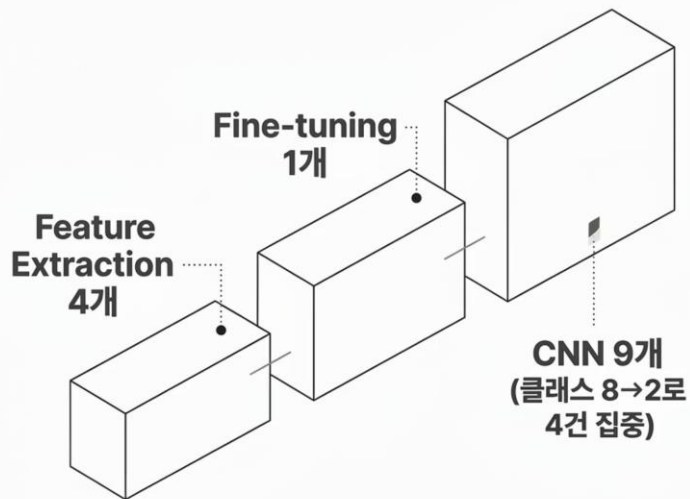


ResNet50-
Feature Extraction



ResNet50-
Fine tuning

오분류 분석 — Confusion Matrix



주요 오분류 사례

CNN — 9개 오분류

클래스 8 → 클래스 2로 4건 집중. 두 랜드마크의 외관이 유사하여 혼동 발생.

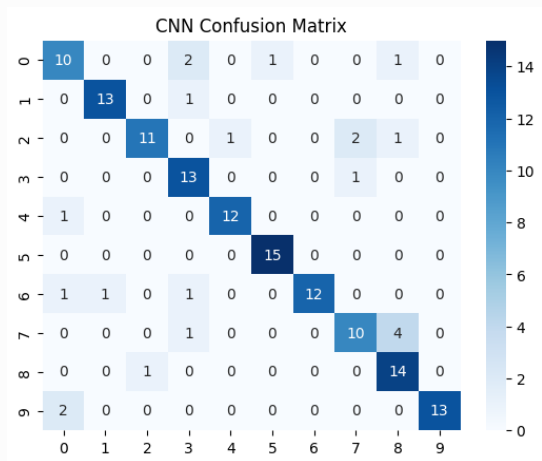
Feature Extraction — 4개

사전학습된 특징 추출 능력으로 유사 클래스 구분력 향상.

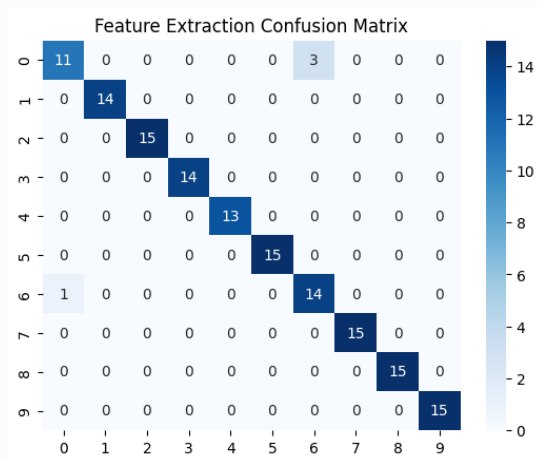
Fine-tuning — 단 1개

전이학습 + 전체 튜닝으로 유사 외관 클래스도 정확히 분류.

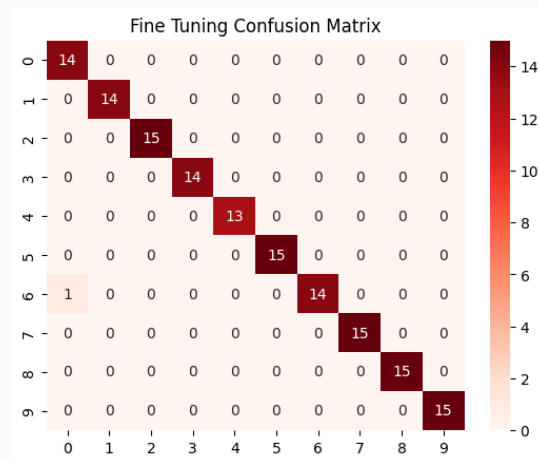
오분류 분석 — Confusion Matrix



CNN



ResNet50-
Fine tuning



ResNet50-
Feature Extraction

어려웠던 점과 해결 방법

Colab ↔ GitHub 연동 오류

! 명령어로 터미널 코드 직접 실행, 생성형 AI로 오류 원인 분석 및 해결

적은 데이터 과적합 우려

RandomHorizontalFlip + RandomRotation(10°) 적용으로 일반화 성능 확보

③ 추가 개선 아이디어: Early Stopping / ColorJitter-RandomErasing / Learning Rate Scheduler

팀원별 AI 활용 내역

박수빈

- Colab ↔ Git 연동 방법 및 오류 해결
- CIFAR-10 연습 커리큘럼 요청 → CNN /전이학습 차이 학습 후 적용
- 그래프 시각화 코드 피드백 및 커스터마이징

김소연

- Colab에서 Git 파일 여는 방법 질문
- 데이터셋에 적합한 optimizer·batch size 추천 요청

김연수

- 프로젝트 전체 과제 요구사항 분석 요청
- CSV 파일 처리 및 DataLoader 구현 방법 질문
- CNN vs 전이학습 개념 비교 정리

팀원 역할 분담 및 핵심 요약

역할 분담

이름	담당	주요 수행 내용
김연수	팀원 A	데이터 구조 분석, CSV 처리, Dataset/DataLoader 구현, 데이터 증강 실험
김소연	팀원 B	CNN 모델 구현, 학습 분석, 하이퍼파라미터 튜닝, pretrained 비교 실험
박수빈	팀원 C	전이학습 구현, 성능 비교, 오분류 분석, 제출 파일 생성, 시각화

핵심 요약

1. Pretrained 가중치가 결정적

적은 데이터 환경에서 사전학습 여부가 성능을 좌우

2. Fine-tuning ResNet18 최고 성능

Val Acc **99.31%** 달성

3. 유사 클래스 오분류 대폭 개선

전이학습으로 외관 유사 랜드마크도 정확히 구분

추후 Early Stopping, 다양한 Augmentation, LR Scheduler 적용 시 추가 개선 가능.

C팀 | 박수빈 · 김연수 · 김소연

이상 발표를 마치겠습니다.